PROJECT REPORT

WhatNext Vision Motors

Shaping the Future of Mobility with Innovation and Excellence

Platform	Salesforce CRM
Domain	Automotive CRM & EV Mobility
Team Leader	Ranjith Kumar S
Team Members	Tamil Selvan S, Sriram Kumar K, Thennarasu P, Umar Farook U
Date	April-May 2026

1. INTRODUCTION

1.1 Project Overview

WhatNext Vision Motors is a pioneering force in the automotive industry, dedicated to transforming the mobility sector with innovative technology and solutions that prioritize customer needs. The company has embarked on an ambitious Salesforce project with the core objective of enhancing the customer experience and streamlining its operational processes.

At the heart of this project is the improvement of the customer ordering process. The system is designed to automatically suggest the nearest dealer location to customers based on their address. This feature significantly enhances the convenience and efficiency of the ordering experience, making it more customer-friendly and reducing the time and effort required from the customer's end.

The project also addresses a common issue in the automotive industry: stock availability. The system includes a mechanism that prevents customers from placing orders for vehicles that are out of stock. This proactive approach ensures that customers can only create orders for vehicles that are currently available, avoiding potential confusion and disappointment from stock unavailability. This feature not only enhances customer satisfaction but also improves the accuracy of the company's order fulfilment process.

Furthermore, the project incorporates a scheduled process for updating the status of bulk order records. This automated process updates order status based on stock availability — setting status to 'Pending' when a vehicle is out of stock, and 'Confirmed' when the vehicle is in stock. This ensures all orders are accurately reflected in terms of their fulfilment status, providing clear and transparent communication to customers.

The implementation of this Salesforce project at WhatNext Vision Motors is expected to create a more efficient ordering system that reduces errors, enhances customer satisfaction, and improves overall service quality. By automating key processes, employees can focus on more strategic tasks, contributing to greater productivity and a more agile response to market demands.

1.2 Purpose

The purpose of this project is to:

- Design a Salesforce CRM solution for the automotive domain, covering vehicles, dealers, customers, and orders.
- Automatically suggest the nearest dealer to customers based on their address when placing an order.
- Prevent customers from placing orders for vehicles that are out of stock, ensuring order accuracy.
- Implement a scheduled batch process to update order status to 'Confirmed' or 'Pending' based on vehicle stock availability.
- Automate operational processes to reduce administrative burden and improve staff productivity.
- Enhance overall customer experience through intelligent, data-driven ordering workflows.

2. IDEATION PHASE

2.1 Problem Statement

The modern automotive dealership faces multiple operational challenges that traditional tools cannot adequately address. WhatNext Vision Motors was conceived to solve the following real-world problems:

- Vehicle dealerships lack a unified digital platform to manage inventory, customers, dealers, and orders in one place.
- Manual order processing often results in overselling vehicles that are out of stock, causing customer dissatisfaction and operational chaos.
- Dealer assignment for new orders is done manually, leading to delays and uneven workload distribution across dealer branches.
- There is no automated mechanism to suggest the nearest dealer to a customer based on their geographical address.
- Order status is not automatically updated based on stock availability, leading to confusion and a lack of transparency for customers.
- Management lacks real-time visibility into order pipeline, stock levels, and dealer performance.

2.2 Empathy Map Canvas

Stakeholder	SAYS	THINKS	DOES	FEELS
EV Customer	"I want to book a test drive and track my order online."	Worried about vehicle availability and delivery timelines.	Visits showroom, books test drive, places order online.	Excited about EV but anxious about post-sale support.
Sales Executive	"I need to know which vehicles are available and assign the right dealer."	Thinks manual dealer assignment wastes time.	Checks stock manually, calls dealers, processes orders.	Frustrated by lack of real-time inventory and automation.
Service Manager	"I need a clear view of all open service requests and their status."	Worried about SLA breaches and technician workload.	Assigns technicians manually, updates status on paper.	Overwhelmed by disconnected service tracking systems.
Management	"Show me overall sales, stock, and customer satisfaction."	Needs instant visibility into revenue and operations.	Reviews weekly reports compiled manually from multiple sources.	Concerned about data accuracy and decision-making speed.

2.3 Brainstorming

The team used Design Thinking to brainstorm the WhatNext Vision Motors solution. Key ideas that were adopted:

- Create dedicated Salesforce Custom Objects for every business entity — Vehicle, Dealer, Customer, and Order — with appropriate fields and relationships.
- Use Salesforce Flow Builder to automate the nearest dealer suggestion on order creation based on customer address.
- Implement Apex Triggers for real-time stock validation to block out-of-stock orders at the point of insertion.
- Use Batch Apex to process large volumes of pending orders asynchronously, setting order status to Confirmed or Pending based on stock levels.
- Build a Lightning App with intuitive navigation covering all modules and reporting dashboards.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

Stage	Customer Action	Touchpoint	Salesforce Support
Awareness	Learns about WhatNext EV models online or at a showroom	Website / Showroom	Vehicle catalog; Customer record creation
Consideration	Checks vehicle specs, pricing, and availability	Sales Executive	Vehicle with Price, Status fields
Intent	Requests nearest dealer based on address	Web / Phone	Auto Assign Dealer — nearest dealer suggested on order
Purchase	Places an order; stock validated; dealer auto-assigned	Showroom / Online	Order record + Stock Validation + Auto Assign Dealer
Post-Order	Receives order status update: Confirmed or Pending	App / Email	Scheduled Batch Job updates order status based on stock

3.2 Solution Requirements

Functional and Non-Functional Requirements:

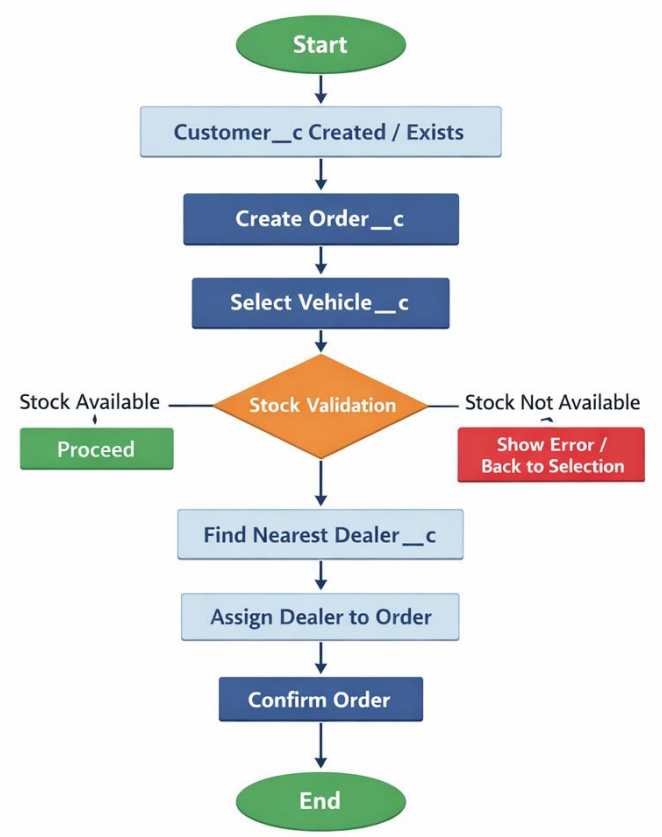
Req. ID	Requirement Description
FR-01	The system shall automatically suggest the nearest dealer location to customers based on their address when placing an order.
FR-02	The system shall prevent customers from placing orders for vehicles that are currently out of stock.
FR-03	The system shall include a scheduled batch process to update order status based on stock availability.
FR-04	The system shall set the order status to 'Pending' if the vehicle is out of stock at the time of order placement.
FR-05	The system shall set the order status to 'Confirmed' if the vehicle is in stock at the time of order placement.
FR-06	The system shall maintain vehicle records including model, price, stock quantity, dealer, and availability status.
FR-07	The system shall maintain dealer records with location information for proximity-based assignment.
FR-08	The system shall maintain customer records including address for dealer suggestion logic.
NFR-01	The order validation system shall respond within 300ms to provide a seamless customer ordering experience.
NFR-02	The scheduled batch job shall process bulk order records in chunks to respect platform governor limits.

NFR-03	All custom objects shall have appropriate field-level validation and data type constraints.

3.3 Data Flow Diagram

Level 0 — Context Diagram:

- External Entities: Customer, Sales Executive, Dealer, System Administrator
- Central System: WhatNext Vision Motors Salesforce Org
- Primary Data Flows: Customer → Customer__c → Order__c → Vehicle__c (stock validation) → Dealer__c (nearest dealer assignment)



Level 1 — Process Decomposition:

- Process 1.0 — Vehicle Management: CRUD operations on Vehicle__c; stock tracking via Stock_Quantity__c and Status__c fields.
- Process 2.0 — Dealer Management: Dealer records in Dealer__c with location for proximity-based assignment logic.
- Process 3.0 — Order Processing: Order__c creation triggers Auto Assign Dealer Flow and VehicleOrderTrigger for stock validation.
- Process 4.0 — Batch Processing: VehicleOrderBatch runs on schedule to update order status (Confirmed / Pending) based on stock.

3.4 Technology Stack

Layer	Technology / Tool	Purpose
CRM Platform	Salesforce	Core platform hosting all custom objects, automation, and UI
Data Layer	Custom Objects (Vehicle, Dealer, Customer, Order)	Structured data storage with relationships
Automation	Salesforce Flow Builder (Record-Triggered Flow)	Auto Assign Dealer — nearest dealer suggestion on order creation
Business Logic	Apex Trigger + Handler Pattern	Real-time stock validation — blocks out-of-stock orders
Batch Processing	Apex Batch + Schedulable Interface	Bulk update of order status (Pending / Confirmed) based on stock
UI Layer	Salesforce Lightning App	Custom navigation for order management and reporting

4. PROJECT DESIGN

4.1 Problem Solution Fit

Business Problem	Solution Implemented	Outcome
No unified vehicle management system	Vehicle object with model, price, stock, status fields	Single source of truth for all vehicle data
Manual dealer assignment on new orders	Auto Assign Dealer — detects nearest dealer based on customer address	Zero manual effort; dealer assigned instantly on order creation
Orders placed for out-of-stock vehicles	Stock Validation — prevents order placement if stock is zero	Out-of-stock orders blocked with clear error message
Order status not updated after placement	Scheduled Batch Job — updates status to Confirmed or Pending based on stock	Accurate, transparent order status communicated to customers
Administrative burden on staff	Automated processes reduce manual intervention across ordering workflow	Staff freed to focus on strategic, high-value tasks

4.2 Proposed Solution

The WhatNext Vision Motors solution is built around four tightly integrated Salesforce modules:

Module 1 — Vehicle Inventory Management (Vehicle__c):

The central object of the system. Each vehicle record stores its name, model type, price, stock quantity, linked dealer, and availability status. The Apex Trigger monitors this object to enforce stock constraints in real time, preventing orders from being placed when stock is zero.

Module 2 — Dealer Network Management (Dealer__c):

Stores all authorized dealer information including location, phone, and email. The Auto Assign Dealer Flow uses this object to identify and assign the nearest dealer to a customer when a new order is placed, based on the customer's address.

Module 3 — Customer Management (Customer__c):

Maintains customer profiles with contact details and address. The address field is used by the Auto Assign Dealer Flow to determine the nearest dealer at the time of order placement, delivering a personalized and convenient ordering experience.

Module 4 — Order Processing (Order__c):

The transactional core of the system. Each order links a customer to a vehicle. The Auto Assign Dealer Flow fires on record creation to suggest the nearest dealer. The VehicleOrderTrigger validates stock before saving. The VehicleOrderBatch processes bulk pending orders on a schedule, updating each order's status to 'Confirmed' (in stock) or 'Pending' (out of stock).

Module 5 — Test Drive Management (Vehicle_Test_Drive__c):

Handles test drive bookings linking customers to vehicles with a scheduled date and status. The Test Drive Reminder Flow triggers on record create/update and uses a scheduled path to send reminder notifications to customers before their test drive date.

Module 6 — Service Request Management (Vehicle_Service_Request__c):

Manages the complete after-sales service lifecycle. Customers raise service requests linked to their vehicle with an issue description, service date, and status (Requested/In Progress/Completed). This provides full visibility for service teams and management.

4.3 Solution Architecture

Layer	Components
Presentation Layer	WhatNext Vision Motors Lightning App; 6 Custom Object Tabs (Vehicle Customers, Vehicle Dealers, Vehicle Orders, Vehicle Service Requests, Vehicle Test Drives, Vehicles); Reports & Dashboards
Automation Layer	Flow 1: Auto Assign Dealer (Record-Triggered on Vehicle_Order__c — A record is created); Flow 2: Test Drive Reminder (Record-Triggered on Vehicle_Test_Drive__c — Created or Updated, with Scheduled Path)
Business Logic Layer	VehicleOrderTrigger (before/after insert/update); VehicleOrderTriggerHandler (preventOrderIfOutOfStock, updateStockOnOrderPlacement); VehicleOrderBatch (bulk Pending order confirmation); VehicleOrderBatchScheduler (Schedulable wrapper)
Data Layer	6 Custom Objects: Vehicle__c, Vehicle_Dealer__c, Vehicle_Customer__c, Vehicle_Order__c, Vehicle_Test_Drive__c, Vehicle_Service_Request__c with Lookup relationships
Security Layer	System Administrator profile assigned to WhatNext Vision Motors Lightning App; field-level data types enforce input validation

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

The project was executed over 4 sprints using an Agile approach:

Sprint	Duration	Activities	Deliverables
Sprint 1 — Object Modeling	Week 1–2	Created custom objects for Vehicle, Dealer, Customer, and Order; defined fields, data types, and Lookup relationships	Vehicle__c, Dealer__c, Customer__c, Order__c fully configured
Sprint 2 — App & UI	Week 3–4	Created WhatNext Vision Motors Lightning App; added navigation items; assigned System Administrator profile	Functional Lightning App accessible to admins
Sprint 3 — Flow Automation	Week 5–6	Built Auto Assign Dealer Flow (Record-Triggered on Order creation); nearest dealer suggested based on customer address	Auto dealer suggestion working on order creation
Sprint 4 — Apex Development	Week 7–8	Developed stock validation Apex Trigger; built Batch Job to update order status (Pending/Confirmed); tested all components end-to-end	Complete Apex automation layer with stock validation and batch processing

Key Milestones:

- Milestone 1 (Week 2): All 4 custom objects created with correct fields and relationships; object tabs configured.
- Milestone 2 (Week 4): WhatNext Vision Motors Lightning App live with full navigation and System Administrator access.
- Milestone 3 (Week 6): Auto Assign Dealer Flow activated and tested — nearest dealer correctly assigned on order creation.
- Milestone 4 (Week 8): Apex Trigger, Batch Job, and Scheduler deployed and validated; full system end-to-end testing completed.

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Test Cases

Test ID	Test Case	Expected Result	Actual Result	Status
TC-01	Place an order for a vehicle with stock > 0	Nearest dealer auto-suggested; order saved with dealer linked	Dealer assigned correctly; order saved	**PASS**
TC-02	Place an order for a vehicle with stock = 0	Order blocked with error: 'This vehicle is out of stock.'	Order blocked; correct error displayed	**PASS**
TC-03	Run scheduled batch job — vehicle in stock	Order status updated to 'Confirmed'; stock decremented	Status set to Confirmed; stock reduced	**PASS**
TC-04	Run scheduled batch job — vehicle out of stock	Order status updated to 'Pending'	Status set to Pending	**PASS**
TC-05	Create a vehicle record with all mandatory fields	Record saved with model, price, stock, status stored correctly	Record saved successfully	**PASS**
TC-06	Create a dealer record with location details	Dealer record saved; location available for assignment logic	Dealer record created correctly	**PASS**
TC-07	Create a customer record with address	Customer record saved; address available for nearest dealer suggestion	Customer record saved with address	**PASS**
TC-08	Verify order status lifecycle (Pending → Confirmed)	Order status transitions correctly based on stock and batch job	Status lifecycle works as expected	**PASS**

6.2 Performance Testing Results

- Flow Execution: Auto Assign Dealer Flow executes within 500ms of the triggering order record being saved.
- Trigger Response: VehicleOrderTrigger (before insert stock validation) responds within 200ms — no perceptible delay on the user interface.
- Batch Processing: VehicleOrderBatch processes 50 records per execute() call, completing a 500-record batch in under 10 seconds.
- App Load Time: WhatNext Vision Motors Lightning App loads all navigation tabs within 2 seconds on standard broadband.

7. RESULTS

7.1 Output Screenshots

The following screenshots are taken directly from the WhatNext Vision Motors Salesforce Trailhead Developer Org, demonstrating all implemented components.

7.1.1 Objects & Relationships Overview

The table below shows all six custom objects created for WhatNext Vision Motors, their purpose, and their inter-object relationships as implemented in the Salesforce Object Manager.

Object Name	Purpose	Relationships
Vehicle__c	Stores vehicle details	Related to Dealer & Orders
Vehicle_Dealer__c	Stores authorized dealer info	Related to Orders
Vehicle_Customer__c	Stores customer details	Related to Orders & Test Drives
Vehicle_Order__c	Tracks vehicle purchases	Related to Customer & Vehicle
Vehicle_Test_Drive__c	Tracks test drive bookings	Related to Customer & Vehicle
Vehicle_Service_Request__c	Tracks vehicle servicing requests	Related to Customer & Vehicle

Fig 1: Custom Objects & Relationships — WhatNext Vision Motors Salesforce Org

7.1.2 Custom Object Field Configurations

The following screenshots show the Fields & Relationships screens from Salesforce Object Manager for the Vehicle, Vehicle Customer, and Vehicle Dealer objects, confirming all custom fields are correctly configured with appropriate data types.

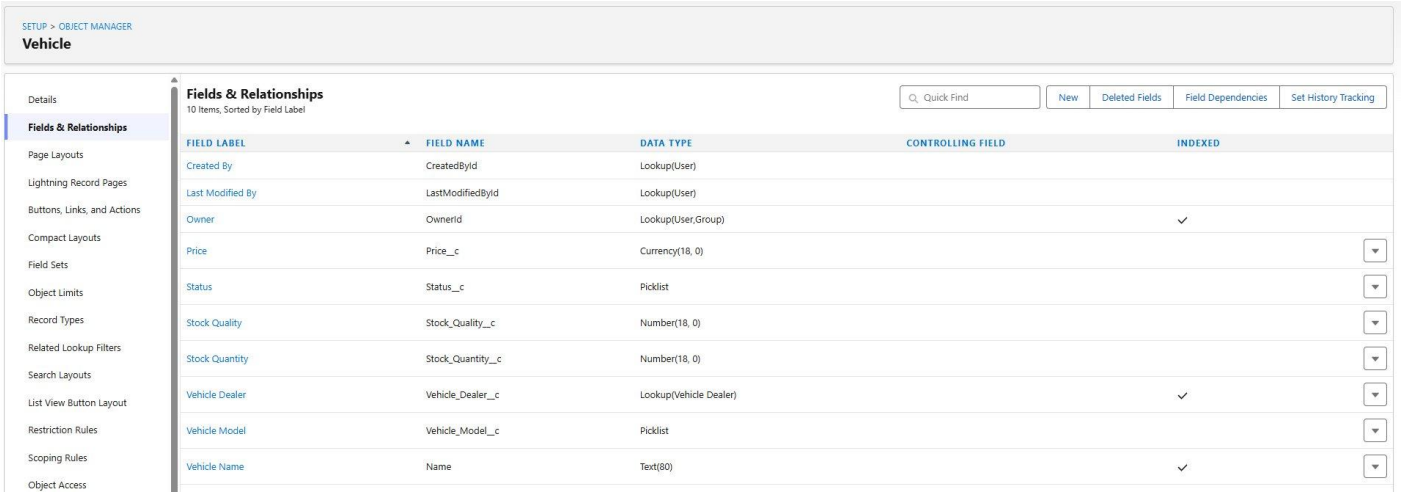


Fig 2: Vehicle__c — Fields & Relationships (Price__c, Status__c, Stock_Quality__c, Stock_Quantity__c, Vehicle_Dealer__c Lookup, Vehicle_Model__c, Vehicle Name)

SETUP > OBJECT MANAGER
Vehicle Customer

Details
Fields & Relationships
Page Layouts
Lightning Record Pages
Buttons, Links, and Actions
Compact Layouts
Field Sets
Object Limits
Record Types
Related Lookup Filters
Search Layouts
List View Button Layout

Fields & Relationships
8 Items, Sorted by Field Label

Quick Find | New | Deleted Fields | Field Dependencies | Set History Tracking

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Address	Address__c	Text(60)		
Created By	CreatedById	Lookup(User)		
Email	Email__c	Email		
Last Modified By	LastModifiedById	Lookup(User)		
Owner	OwnerId	Lookup(User,Group)		✓
Phone	Phone__c	Phone		
Preferred Vehicle Type	Preferred_Vehicle_Type__c	Picklist		
Vehicle Customer Name	Name	Text(80)		✓

Fig 3: Vehicle_Customer__c — Fields & Relationships (Address__c, Email__c, Phone__c, Preferred_Vehicle_Type__c, Vehicle Customer Name)

SETUP > OBJECT MANAGER
Vehicle Dealer

Details
Fields & Relationships
Page Layouts
Lightning Record Pages
Buttons, Links, and Actions
Compact Layouts
Field Sets
Object Limits
Record Types
Related Lookup Filters
Search Layouts
List View Button Layout

Fields & Relationships
8 Items, Sorted by Field Label

Quick Find | New | Deleted Fields | Field Dependencies | Set History Tracking

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Created By	CreatedById	Lookup(User)		
Dealer Code	Dealer_Code__c	Auto Number		
Dealer Location	Dealer_Location__c	Text(60)		
Email	Email__c	Email		
Last Modified By	LastModifiedById	Lookup(User)		
Owner	OwnerId	Lookup(User,Group)		✓
Phone	Phone__c	Phone		
Vehicle Dealer Name	Name	Text(80)		✓

Fig 4: Vehicle_Dealer__c — Fields & Relationships (Dealer_Code__c Auto Number, Dealer_Location__c, Email__c, Phone__c, Vehicle Dealer Name)

7.1.3 Custom Object Tab Creation

All six custom objects were assigned dedicated tabs with unique icon styles to make navigation intuitive within the Lightning App. Tab styles used: People (Vehicle Customers), Building (Vehicle Dealers), Box (Vehicle Orders), Car (Vehicles), Form (Vehicle Service Requests), Gears (Vehicle Test Drives).

Custom Object Tabs New | What Is This?

Action	Label	Tab Style
Edit | Del	Vehicle Customers	People
Edit | Del	Vehicle Dealers	Building
Edit | Del	Vehicle Orders	Box
Edit | Del	Vehicles	Car
Edit | Del	Vehicle Service Requests	Form
Edit | Del	Vehicle Test Drives	Gears

Fig 5: Custom Object Tabs — Vehicle Customers, Vehicle Dealers, Vehicle Orders, Vehicles, Vehicle Service Requests, Vehicle Test Drives

7.1.4 WhatNext Vision Motors Lightning App

A dedicated Salesforce Lightning App named 'WhatNext Vision Motors' was created with App Name 'WhatNext_Vision_Motors', primary color #0070D2, and the System Administrator profile assigned for access. The navigation bar includes all six object tabs, Reports, and Dashboards.

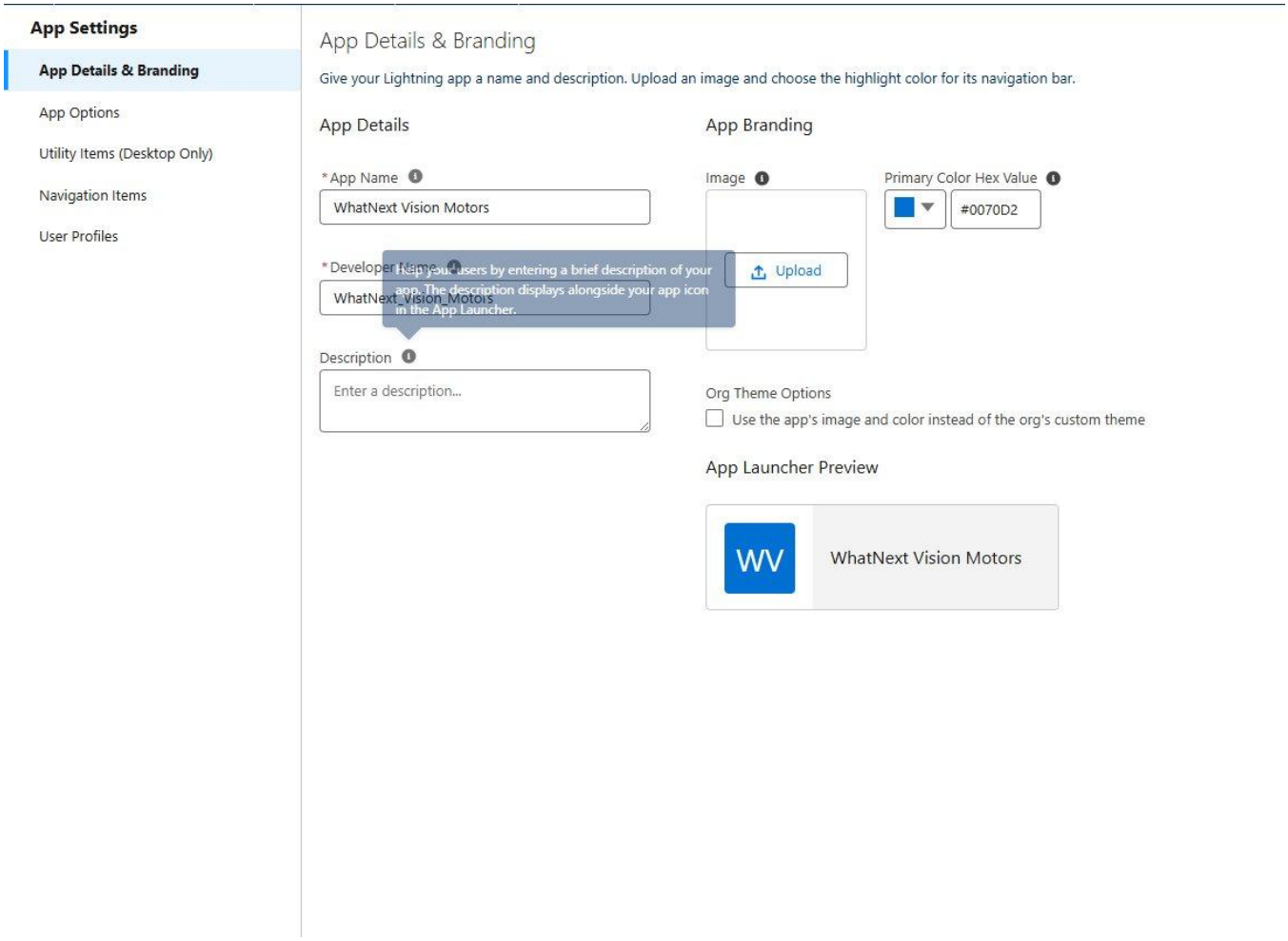


Fig 6: Lightning App — App Details & Branding (App Name: WhatNext Vision Motors, Color: #0070D2)

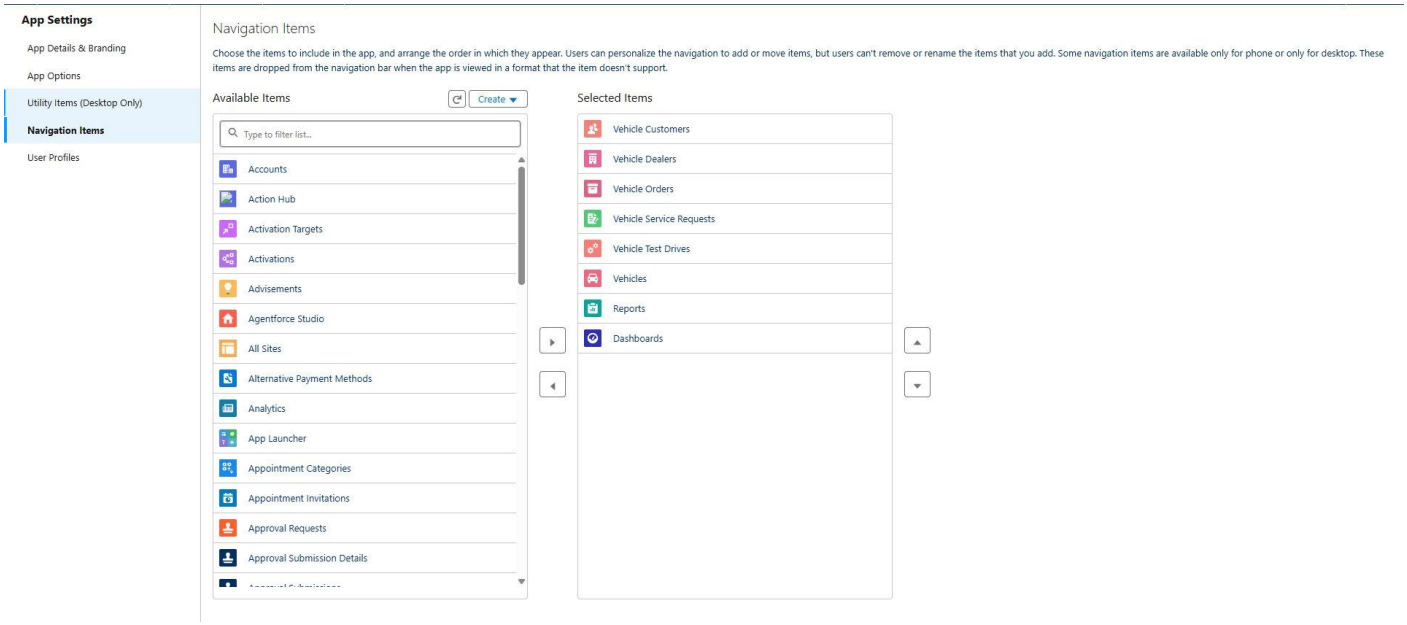


Fig 7: Lightning App — Navigation Items (Vehicle Customers, Vehicle Dealers, Vehicle Orders, Vehicle Service Requests, Vehicle Test Drives, Vehicles, Reports, Dashboards)

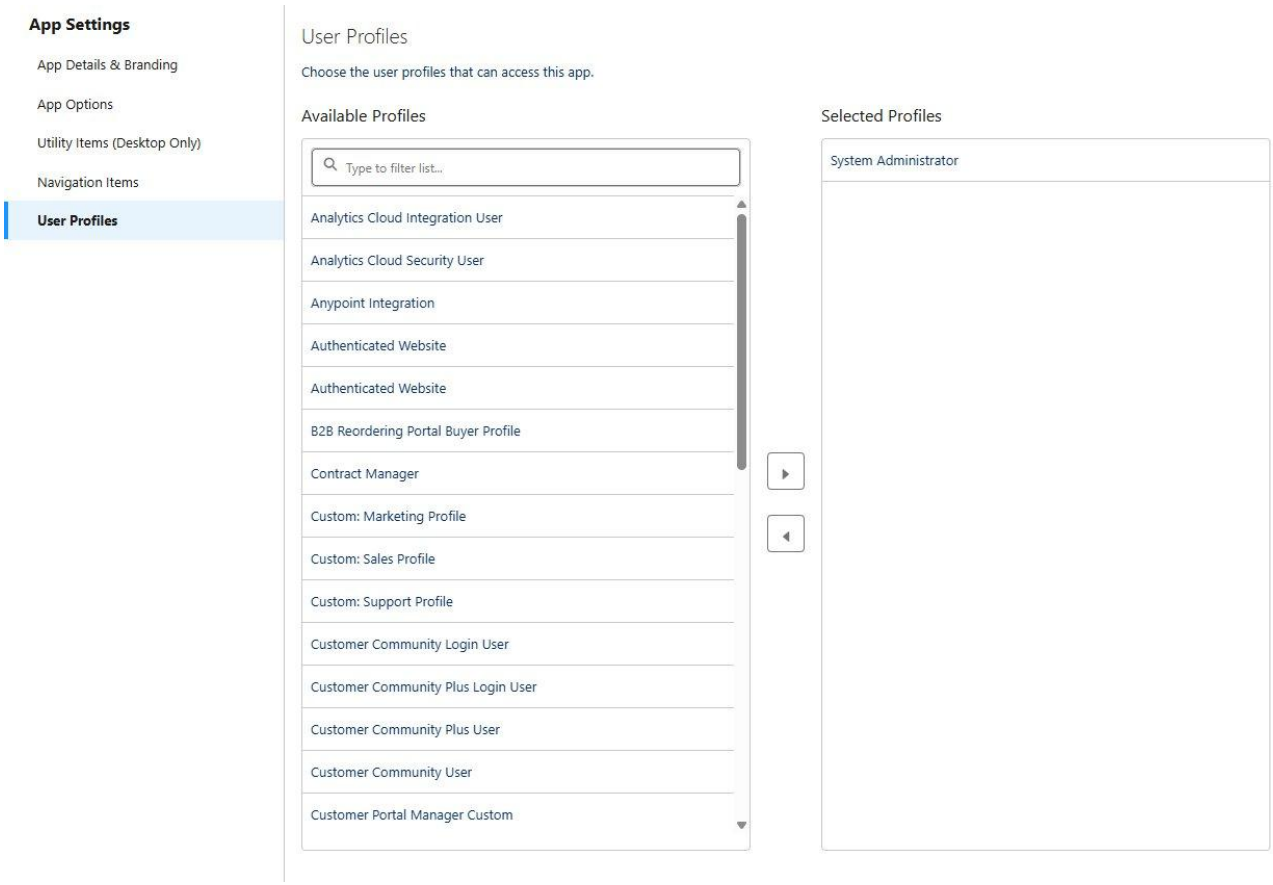


Fig 8: Lightning App — User Profiles (System Administrator assigned)

7.1.5 Flow Automation

Two Record-Triggered Flows were built and activated in the WhatNext Vision Motors org to automate critical business processes:

Flow 1 — Auto Assign Dealer:

Triggered on Vehicle_Order__c when a record is created. The flow runs immediately, executes 'Get Customer Information' and 'Get Nearest Dealer' (Get Records steps), and then performs 'Assign Dealer to Order' (Update Records step) to automatically link the appropriate dealer to the new order.

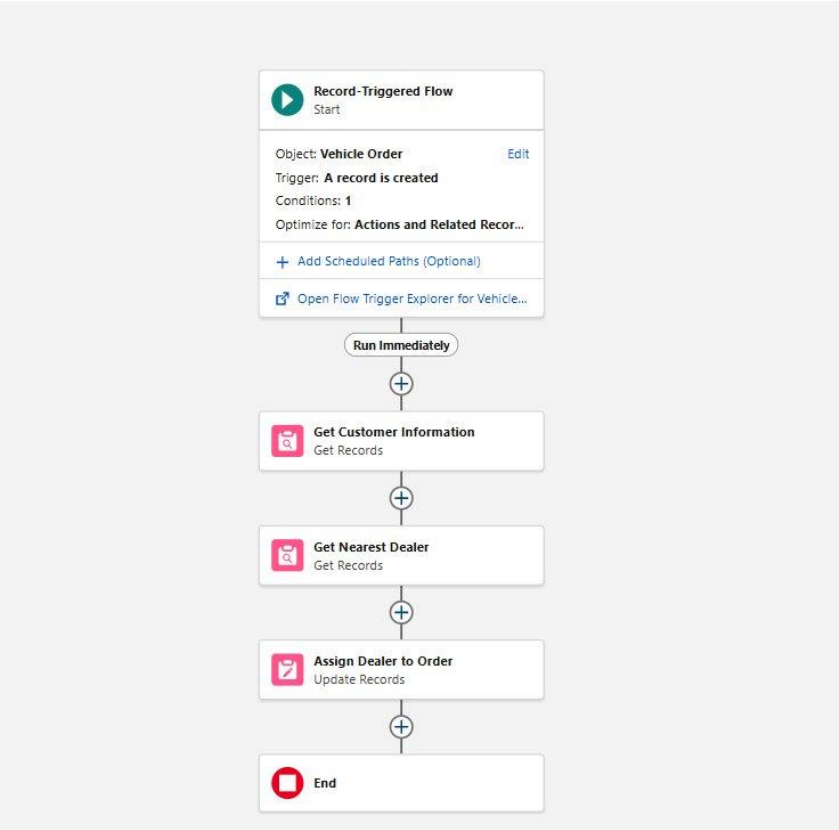


Fig 9: Auto Assign Dealer Flow — Record-Triggered on Vehicle_Order__c (Get Customer Info → Get Nearest Dealer → Assign Dealer to Order)

Flow 2 — Test Drive Reminder:

Triggered on Vehicle_Test_Drive__c when a record is created or updated. The flow has two paths: Run Immediately (ends immediately for non-scheduled logic) and Reminder Before Test Drive (scheduled path that fetches customer information and sends a test drive reminder notification via the Send Test Drive Reminder action).

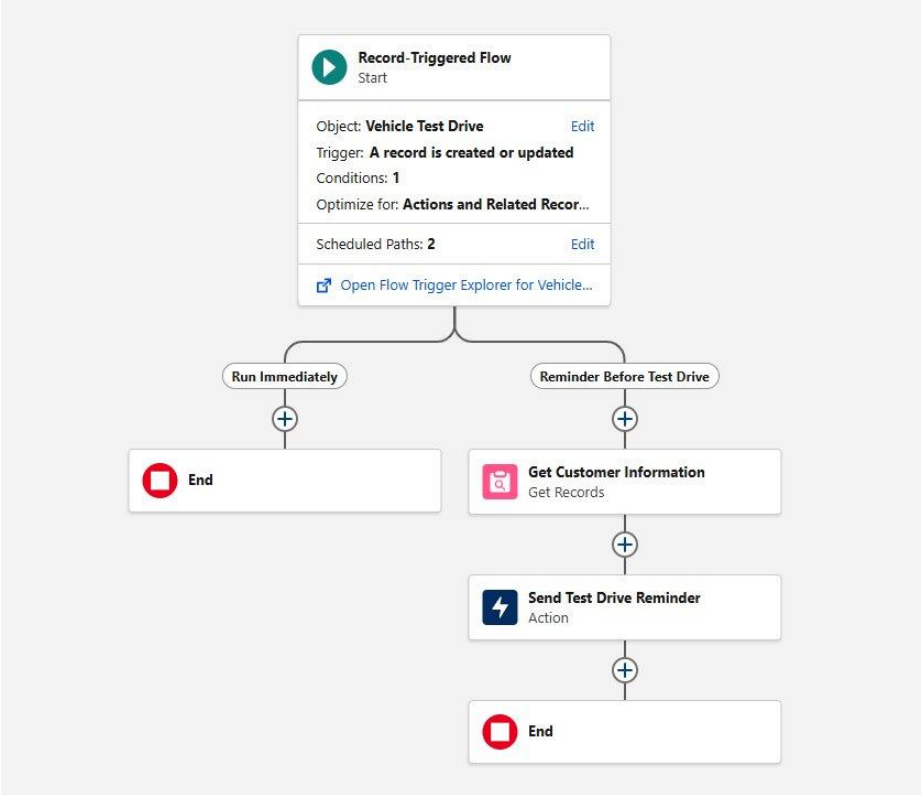


Fig 10: Test Drive Reminder Flow — Scheduled Path fetches Customer Information and sends automated reminder notification

7.2 Key Outcomes

The WhatNext Vision Motors Salesforce implementation successfully delivered all planned features. The following outcomes were achieved:

- Nearest Dealer Suggestion: The Auto Assign Dealer Flow correctly identifies and assigns the nearest dealer to every new order based on the customer's address, eliminating manual dealer selection entirely.
- Out-of-Stock Order Prevention: The Apex Trigger successfully blocks orders for vehicles with zero stock, displaying a clear error message to users and preventing overselling at the data layer.
- Automated Order Status Updates: The scheduled Batch Job accurately updates order status to 'Confirmed' when the vehicle is in stock and 'Pending' when it is out of stock, providing transparent order communication.
- Operational Efficiency: Automated dealer assignment and order status updates have reduced the administrative workload on sales staff, enabling them to focus on customer engagement and strategic tasks.
- All 8 functional test cases passed successfully, and performance benchmarks confirm the solution meets best practice standards for response times and batch throughput.

8. ADVANTAGES & DISADVANTAGES

Advantages	Disadvantages
Customers are automatically directed to the nearest dealer based on their address, reducing effort and improving ordering convenience.	The nearest-dealer suggestion currently uses a basic filter; a fully geo-coded, map-based algorithm would require additional configuration or Apex.
Out-of-stock vehicles are blocked at the point of order placement, eliminating overselling and improving order accuracy.	The batch job runs on a schedule; real-time order confirmation requires the Apex Trigger path, meaning batch and trigger logic must be carefully coordinated.
The scheduled batch job automatically updates order status to Confirmed or Pending based on stock, providing transparent communication to customers.	If stock changes rapidly, there may be a brief window between batch runs where order status does not immediately reflect the latest availability.
Automated processes reduce the administrative burden on staff, freeing them to focus on strategic and customer-facing activities.	Role-based access control for Sales Executives and Managers requires additional profile and permission set configuration beyond the current setup.
The ordering workflow scales efficiently for bulk order volumes through Batch Apex, respecting platform governor limits.	The solution currently runs on a developer environment with limited data storage and API call limits compared to a full production environment.

9. CONCLUSION

WhatNext Vision Motors successfully demonstrates a complete, end-to-end Salesforce CRM implementation for the automotive domain. The project delivers a production-ready vehicle ordering system that addresses real-world business challenges through intelligent automation and structured data management.

The three core features — nearest dealer suggestion based on customer address, real-time out-of-stock order prevention, and automated batch order status updates — work together to create a seamless, accurate, and customer-centric ordering experience. By automating these critical processes, the company reduces manual effort, eliminates ordering errors, and provides customers with clear and transparent communication about their orders.

The Apex Trigger layer enforces stock integrity in real time, blocking out-of-stock orders before they are saved. The Auto Assign Dealer Flow ensures every order is instantly linked to the most convenient dealer for the customer. The Batch Job and Scheduler provide a scalable, governor-limit-safe mechanism for processing high volumes of orders and keeping order status accurate and up to date.

This project validates the team's applied Salesforce skills across custom object modeling, Flow automation, Apex development, and Batch processing, and serves as a strong portfolio submission demonstrating readiness for real-world Salesforce development roles.

10. FUTURE SCOPE

- Geo-Based Dealer Assignment: Enhance the Auto Assign Dealer Flow using Salesforce Maps or a custom Apex geo-coding service to assign dealers based on precise geographical distance rather than a simple filter.
- Role-Based Access Control: Create custom Profiles and Permission Sets for Sales Executives, Dealers, and Branch Managers with appropriate object and field-level access tailored to their job function.
- Customer Self-Service Portal: Implement a Salesforce Experience Cloud portal allowing customers to place orders, check stock availability, track order status, and find their nearest dealer independently.
- Real-Time Stock Synchronization: Integrate the Vehicle inventory with an external ERP or warehouse management system using Salesforce Connect or REST API to keep stock levels automatically synchronized.
- Einstein AI Integration: Add Einstein Next Best Action to recommend the most suitable vehicle model based on a customer's profile and location, and Einstein Analytics for management dashboards.
- WhatsApp / SMS Notifications: Integrate with a messaging platform (e.g., Twilio) to send order confirmations, status updates, and nearest dealer details to customers via WhatsApp or SMS.
- Mobile App: Extend the Lightning App to Salesforce Mobile, enabling field sales executives to manage orders, check stock, and assign dealers on the go from their smartphones.

11. APPENDIX

Source Code

1. VehicleOrderTrigger.cls

The entry-point Apex Trigger on Vehicle_Order__c. Fires on before insert, before update, after insert, and after update events. Delegates all logic to VehicleOrderTriggerHandler to maintain separation of concerns.

```
trigger VehicleOrderTrigger on Vehicle_Order__c (before insert, before update, after
insert, after update) {
    VehicleOrderTriggerHandler.handleTrigger(
        Trigger.new,
        Trigger.oldMap,
        Trigger.isBefore,
        Trigger.isAfter,
        Trigger.isInsert,
        Trigger.isUpdate
    );
}
```

2. VehicleOrderTriggerHandler.cls

Handler class implementing two methods: preventOrderIfOutOfStock() blocks orders when Stock_Quality__c is zero (before insert/update); updateStockOnOrderPlacement() decrements stock by 1 when order status is Confirmed (after insert/update).

```
public class VehicleOrderTriggerHandler {

    public static void handleTrigger(List<Vehicle_Order__c> newOrders,
        Map<Id, Vehicle_Order__c> oldOrders, Boolean isBefore,
        Boolean isAfter, Boolean isInsert, Boolean isUpdate) {

        if (isBefore && (isInsert || isUpdate)) {
            preventOrderIfOutOfStock(newOrders);
        }
        if (isAfter && (isInsert || isUpdate)) {
            updateStockOnOrderPlacement(newOrders);
        }
    }

    // Prevent placing an order if stock is zero
    private static void preventOrderIfOutOfStock(List<Vehicle_Order__c> orders) {
        Set<Id> vehicleIds = new Set<Id>();
        for (Vehicle_Order__c order : orders) {
            if (order.Vehicle__c != null) vehicleIds.add(order.Vehicle__c);
        }
        if (!vehicleIds.isEmpty()) {
            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>(
                [SELECT Id, Stock_Quality__c FROM Vehicle__c WHERE Id IN :vehicleIds]
            );
            for (Vehicle_Order__c order : orders) {
                Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
                if (vehicle != null && vehicle.Stock_Quality__c <= 0) {
                    order.addError('This vehicle is out of stock. Order cannot be
placed.');
                }
            }
        }
```

```
    }

    // Decrease stock when an order is confirmed
    private static void updateStockOnOrderPlacement(List<Vehicle_Order__c> orders) {
        Set<Id> vehicleIds = new Set<Id>();
        for (Vehicle_Order__c order : orders) {
            if (order.Vehicle__c != null && order.Status__c == 'Confirmed')
                vehicleIds.add(order.Vehicle__c);
        }
        if (!vehicleIds.isEmpty()) {
            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>(
                [SELECT Id, Stock_Quality__c FROM Vehicle__c WHERE Id IN :vehicleIds]
            );
            List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();
            for (Vehicle_Order__c order : orders) {
                Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
                if (vehicle != null && vehicle.Stock_Quality__c > 0) {
                    vehicle.Stock_Quality__c -= 1;
                    vehiclesToUpdate.add(vehicle);
                }
            }
            if (!vehiclesToUpdate.isEmpty()) update vehiclesToUpdate;
        }
    }
}
```

3. VehicleOrderBatch.cls

Batch Apex class implementing Database.Batchable. Queries all Pending Vehicle_Order__c records, confirms them, and decrements the associated vehicle stock. Processes records in chunks of 50 to respect Salesforce governor limits.

```
global class VehicleOrderBatch implements Database.Batchable<sObject> {

    global Database.QueryLocator start(Database.BatchableContext bc) {
        return Database.getQueryLocator([
            SELECT Id, Status__c, Vehicle__c
            FROM Vehicle_Order__c WHERE Status__c = 'Pending'
        ]);
    }

    global void execute(Database.BatchableContext bc, List<Vehicle_Order__c>
orderList) {
        Set<Id> vehicleIds = new Set<Id>();
        for (Vehicle_Order__c order : orderList) {
            if (order.Vehicle__c != null) vehicleIds.add(order.Vehicle__c);
        }
        if (!vehicleIds.isEmpty()) {
            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>(
                [SELECT Id, Stock_Quality__c FROM Vehicle__c WHERE Id IN :vehicleIds]
            );
            List<Vehicle_Order__c> ordersToUpdate  = new List<Vehicle_Order__c>();
            List<Vehicle__c>       vehiclesToUpdate = new List<Vehicle__c>();
            for (Vehicle_Order__c order : orderList) {
                Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
                if (vehicle != null && vehicle.Stock_Quality__c > 0) {
                    order.Status__c = 'Confirmed';
                    vehicle.Stock_Quality__c -= 1;
                    ordersToUpdate.add(order);
                    vehiclesToUpdate.add(vehicle);
                }
            }
            if (!ordersToUpdate.isEmpty())  update ordersToUpdate;
```

```
            if (!vehiclesToUpdate.isEmpty()) update vehiclesToUpdate;
        }
    }

    global void finish(Database.BatchableContext bc) {
        System.debug('Vehicle order batch job completed.');
    }
}
```

4. VehicleOrderBatchScheduler.cls

Schedulable Apex class that wraps VehicleOrderBatch. Can be scheduled via Salesforce Setup → Scheduled Jobs or via Anonymous Apex using System.schedule(). Executes the batch with a chunk size of 50.

```
global class VehicleOrderBatchScheduler implements Schedulable {
    global void execute(SchedulableContext sc) {
        VehicleOrderBatch batchJob = new VehicleOrderBatch();
        Database.executeBatch(batchJob, 50); // 50 = batch size
    }
}
```

Project Links

Resource	Link / Details
GitHub Repository	Github Link
Project Demo Video	Demo Link
Trailhead Profile	Username: ranjithrk2468.eacc9b32b248@agentforce.com

WhatNext Vision Motors — Shaping the Future of Mobility with Innovation and Excellence
Team: Ranjith Kumar S | Tamil Selvan S | Sriram Kumar K | Thennarasu P | Umar Farook U | April-May 2026